Progress Report – Sudoku CSP Solver

Prateek Das - 002510704

Instructor – Prof. Rose Sloan

Work Completed So Far

So far, I have implemented the core components needed for my project, which models Sudoku as a constraint satisfaction problem (CSP) and compares different search strategies for solving it efficiently.

First, I implemented a Sudoku board representation in Python that supports parsing puzzles, representing the grid internally, and checking whether number placements satisfy Sudoku constraints. In this formulation, each grid cell represents a variable, the domain consists of the numbers 1–9, and the constraints enforce the Sudoku rules across rows, columns, and 3×3 subgrids.

I then implemented a baseline backtracking solver that recursively fills empty cells while ensuring that each assignment satisfies the constraints. This solver serves as a baseline that more advanced techniques can later be compared against.

Next, I added the Minimum Remaining Values (MRV) heuristic for variable selection. Instead of choosing the first empty cell, the solver selects the cell with the smallest number of valid values. Initial testing shows that this heuristic significantly reduces the search effort compared to naive backtracking.

To support later evaluation, I also added performance metrics that track the number of recursive calls, the number of backtracking steps, and the total runtime required to solve a puzzle. The codebase has been organized into a structured Python project and uploaded to a GitHub repository.

Immediate Next Steps

The next step is to implement forward checking as a constraint propagation technique. After assigning a value to a cell, forward checking removes that value from the domains of neighboring cells that share a row, column, or subgrid. This should allow the solver to detect conflicts earlier and reduce unnecessary search.

After implementing forward checking, I plan to test solver configurations that combine different techniques, such as backtracking with MRV and backtracking with both MRV and forward checking.

Challenges and Adjustments

One challenge so far has been designing the solver structure so that additional techniques such as forward checking can be integrated cleanly. The initial implementation recalculated domains when needed, but forward checking requires maintaining domain information throughout the search. I have started modifying the solver to support this.

At this point, I do not expect any major changes to the overall project scope.

Plan for the Remaining Timeline

Over the next month, I will complete the implementation of forward checking and evaluate the solver on multiple Sudoku puzzles of varying difficulty. I will collect metrics such as recursive calls, backtracking steps, and runtime to compare the performance of different solving strategies. These results will be analyzed and included in the final project report.